Tipos Genéricos em Java

Renato Cerqueira
Departamento de Informática - PUC-Rio

Por que Tipos Genéricos?

```
Map animais  = new HashMap();
List frutas = new ArrayList();

animais.put ("Macaco", new Fruta("Laranja"));  // OK
frutas.add (new Animal("Macaco"));  // OK
Animal animal =
     (Animal)animais.get("Macaco"); // ERRO DE EXECUÇÃO
Fruta fruta = (Fruta)frutas.get(0); // ERRO DE EXECUÇÃO

/* Percorrendo a coleção */
for (Iteraror it = frutas.iterator(); it.hasNext();) {
  Object o = it.next();
  if (o instanceof Fruta) {
    Fruta f = (Fruta)o;
    System.out.println ("Árvore de " + f + " é " +
                        f.getTree());
  }
}
```

Usando Coleções como Tipos Genéricos

```
Map<String, Animal> animais =
     new HashMap<String, Animal>();
List<Fruta> frutas = new ArrayList<Fruta>();

animais.put(“Vaca", new Fruta(”Maçã")); // ERRO DE COMPILAÇÃO
frutas.add(new Animal(”Vaca")); // ERRO DE COMPILAÇÃO
animais.put("Macaco", new Animal("Macaco")); // OK
frutas.add (new Fruta("Laranja")); // OK
Animal animal = animais.get("Macaco"); // OK
Fruta fruta = frutas.get(0); // OK

/* Usando o for estendido para percorrer a coleção */
for (Fruta f : frutas) {
  System.out.println ("Árvore de " + f + " é " +
                      f.getTree());
}
```

Vantagens

- **Simplicidade**:

  Não é necessário *Type Casting* para extrair os objetos das coleções

- **Robustez**:

  O compilador não permite colocar na coleção elementos incompatíveis com os tipos declarados

Tipo Genérico

- Um **tipo genérico** (*generic type*) é um tipo que possui um ou mais **parâmetros de tipo**.

```
public interface List<E> {
    void add (E element);
    Iterator<E> iterator();
}
```

```
public interface Map<K,V> {
    void put (K key, V value);
    V get (K key);
}
```

Parâmetros de Tipo

- Os parâmetros de tipo de um tipo genérico podem ser ou não **limitados**.
- O limite de um parâmetro de tipo restringe os tipos que podem ser usados como argumento.
- O parâmetro de tipo limitado dá acesso aos métodos do “tipo limite”.

Parâmetro de Tipo Não Limitado

```
class Pair<X,Y>  {
  private X first;
  private Y second;
  public Pair(X a1, Y a2) {
    first  = a1;
    second = a2;
  }
  public X getFirst()  { return first; }
  public Y getSecond() { return second; }
  public void setFirst(X arg)  {
    first = arg;
  }
  public void setSecond(Y arg) {
    second = arg;
  }
}
```

Parâmetro de Tipo com Limite

```
class Pair<A extends Animal,
           B extends Integer> {
 ....
}
```

```
class Pair<A extends Comparable<A> & Cloneable,
           B extends Comparable<B> & Cloneable>
implements Comparable<Pair<A,B>>, Cloneable {
 ....
}
```

Limitando os Parâmetros de Tipos

```
<TypeParameter extends Class &
  Interface1 &
   ... &
   InterfaceN>
```

- Uma lista de limites consiste de uma classe e/ou várias interfaces.

Instanciando um Tipo Genérico

- Um **tipo parametrizado** (*parametrized type*) é a instanciação de um tipo genérico usando **argumentos de tipo**.

```
List<String> list = new ArrayList<String>();
```

```
Map<String, Integer> =
    new HashMap<String, Integer> ();
```

Instanciação usando Classes Concretas

```
public void printPair(Pair<String,Long> pair) {
  System.out.println("("+pair.getFirst()+
    ”,"+pair.getSecond()+")") ;
}

Pair<String,Long> limit =
new Pair<String,Long>("maximum",1024L);

printPair(limit);
```

Instanciação usando Curingas

- ? denota um conjunto de tipos

```
public void printPair(Pair<?,?> pair) {
    System.out.println("("+pair.getFirst()+
   ","+pair.getSecond()+")") ;
}

Pair<?,?> limit =
  new Pair<String,Long>("maximum",1024L);

printPair(limit);
```

Impondo Limites aos Curingas

- Pode-se impor limites aos curingas
- Limites superiores: **<? extends T>**
  - Aceita T e todos os seus descendentes
  - Ex: se T for Collection, aceita List, Set, ArrayList, etc.
- Limites inferiores: **<? super T>**
  - Aceita T e todos os seus ascendentes
  - Ex: se T for ArrayList, aceita List, Collection, Object

Curinga com Limite Superior

```
class Canvas {
  public void drawAll(List<? extends Shape> shapes) {
     for(Shape s: shapes) {
      s.draw(this);
    }
  }
}
```

Instanciação usando *Raw Types*

- Um tipo genérico instanciado sem argumentos de tipos.
- Compatibilidade com código legado.
- Deve ser evitado!

```
List l = new ArrayList();
l.add (“Maria”);
l.add (new Integer(2));
l.add (4L);
```

Subtipos

- O código abaixo está correto?

```
List<String> ls = new ArrayList<String>;
List<Object> lo = ls;
```

Subtipos

- O código abaixo está correto?

```
List<String> ls = new ArrayList<String>();
List<Object> lo = ls;

// Se estiver correto, eu posso fazer:
lo.add (new Integer(5));
String s = ls.get(0);
```

Subtipos

- O código abaixo está correto?

```
List<String> ls = new ArrayList<String>();
List<Object> lo = ls;


// Se estiver correto, eu posso fazer:
lo.add (new Integer(5));
String s = ls.get(0); // Erro atribuindo um Integer a um String!
```

Subtipos

- List<String> **não** é subtipo de List<Object>

```
List<String> ls = new ArrayList<String>();
List<Object> lo = ls; // ERRO DE COMPILAÇÃO
```

- Se o tipo F é sub-tipo de B e G é um tipo genérico, G<F> **não** é subtipo de G<B>

Supertipo Genérico

- G<?> é supertipo de G<F> e de G<B>
- Atenção:

```
void printCollection (Collection<?> c) {
    for (Object e : c) {
        System.out.println(e);
    }
}

Collection<?> c = new ArrayList<String>();
c.add (new Object());     // ERRO DE COMPILAÇÃO
c.add ("Maria");          // ERRO DE COMPILAÇÃO
c.add (null);             // OK
```

Métodos Genéricos

- Métodos estáticos, não estáticos e construtores podem ter parâmetros de tipo.

- Usados quando existem **dependências** entre o tipo de retorno e os parâmetros

Exemplo de Método Genérico

```
class Collections {
  public static <A extends Comparable<A>>
  A max(Collection<A> xs) {
     Iterator<A> xi = xs.iterator();
     A w = xi.next();
     while (xi.hasNext()) {
       A x = xi.next();
       if (w.compareTo(x) < 0) w = x;
     }
     return w;
  }
}
```

Invocando Métodos Genéricos

- Os argumentos dos tipos nas chamadas a métodos genéricos quase sempre são inferidos automaticamente.

```
final class Test {
  public static void main (String[ ] args) {
     LinkedList<Long> list =
       new LinkedList<Long>();
     list.add(0L);
     list.add(1L);
     Long y = Collections.max(list);
  }
}
```

Métodos Genéricos vs. Curingas

- Pode-se implementar com métodos genéricos o que se implementa com curingas.

```
interface Collection<E> {
  public boolean containsAll(Collection<?> c);
  public boolean addAll(Collection<? extends E> c);
}
```

```
interface Collection<E> {
  public <T>  boolean containsAll(Collection<T> c);
  public <T extends E>  boolean addAll(Collection<T> c);
}
```

Métodos Genéricos vs. Curingas

- Questão:
  - Quando usar métodos genéricos?
  - Quando usar curingas?
- Se **não** existir dependência entre parâmetros e/ou tipos de retorno, deve-se preferir curingas

Apagamento

- O compilador Java cria uma única representação de byte code para cada tipo genérico ou método genérico.
- Todas as instanciações do tipo genérico e do método genérico são mapeadas para essa representação usando uma técnica chamada de Apagamento (*Type Erasure)*.

Antes do Apagamento

```
class Pair<X,Y>  {
  private X first;
  private Y second;
  public Pair(X a1, Y a2) {
    first  = a1; second = a2;
  }
  public X getFirst()  { return first; }
  public Y getSecond() { return second; }
  public void setFirst(X arg)  { first = arg; }
  public void setSecond(Y arg) { second = arg; }
  public static void main(String[] args) {
    Pair<String,Long> pair = new
        Pair<String,Long>("limit", 10000L);
    String s = pair.getFirst();
    Long   l = pair.getSecond();
    Object o = pair.getSecond();
  }
}
```

Depois do Apagamento

```
class Pair  {
  private Object first;
  private Object second;
  public Pair(Object a1, Object a2) {
    first  = a1; second = a2;
  }
  public Object getFirst()  { return first; }
  public Object getSecond() { return second; }
  public void setFirst(Object arg)  { first = arg; }
  public void setSecond(Object arg) { second = arg; }
  public static void main(String[] args) {
    Pair pair = new Pair("limit", 10000L);
    String s = (String)pair.getFirst();
    Long   l = (Long)pair.getSecond();
    Object o = pair.getSecond();
  }
}
```

Unchecked Warning pelo uso de Raw Types

- O compilador não possui as informações disponíveis para fazer a verificação de tipo

```
TreeSet set = new TreeSet();
set.add (“abc”); // UNCHECKED WARNING
```

```
warning: [unchecked] unchecked call
to add(E) as a member of the raw type
java.util.TreeSet
set.add("abc");
```

Unchecked Warning pelo uso de Casting

```
class Wrapper<T> {
  private T wrapped;
  ...
  public boolean equals(Object other) {
   Wrapper<T> otherWrapper =
     (Wrapper<T>)other; // UNCHECKED WARNING
   return (this.wrapped.equals(otherWrapper.wrapped));
  }
}
```

Efeito Colateral

```
void m1() {
  List<Date> list = new ArrayList<Date>();
  ...
  m2(list);
}
void m2(Object arg) {
  List<String> list =
    (List<String>) arg; // UNCHECKED WARNING


  ...
  m3(list);
}
void m3(List<String> list) {
  String s = list.get(0); // ClassCastException
  ...
}
```

Evitando Unchecked Warning no Casting

```
class Wrapper<T> {
  private T wrapped;
  ...
  public boolean equals(Object other) {
   Wrapper<?> otherWrapper = (Wrapper<?>)other;
   return (this.wrapped.equals(otherWrapper.wrapped));
  }
}
```

É possível evitar todos os unchecked warnings?

- Sim, no caso de uso de *raw types*, quando o programador pode alterar o código legado.

- Não, em alguns casos de casting.

Exemplo

```
@SuppressWarnings("unchecked")
public List<Category> getCategories() {
    checkPermission("any","getCategories");
    PrivilegedExceptionAction<List<Category>> action =
      new PrivilegedExceptionAction<List<Category>>(){
      public List<Category> run () throws Exception {
        return theService.getCategories();
      }
    };
    return (List<Category>)Subject.doAs(subject,
            action);
}
```

Exception Handling

- Não é permitido definir um tipo genérico que herde direta ou indiretamente da classe `Throwable`.
- Consequentemente, não aparece tipos parametrizados no tratamento de exceções.

Arrays

- Não se pode criar arrays cujos componentes sejam tipos parametrizados concretos.
- Podem ser criados arrays de *raw types*, arrays de tipos parametrizados com curingas mas o melhor mesmo é usar coleções de tipos parametrizados concretos.

Declaração e Criação de Arrays

```
Pair<String,String>[] arr = null; // OK
arr = new Pair<String,String>[2]; // ERRO DE COMPILAÇÃO
```

```
Pair<?,?>[] arrayOfPair = new Pair<?,?>[2];  // OK
arrayOfPair[0] = new Pair<String, String>(); // OK
Pair<String, String> first =
          arrayOfPair[0]; // ERRO DE COMPILAÇÃO
```

```
Pair[] arrayOfPair = new Pair[2];  // OK
arrayOfPair[0] = new Pair<String, String>();  // OK
Pair<String, String> first =
          arrayOfPair[0]; // UNCHECKED WARNING
```

Alternativas para Arrays de Tipos Genéricos

```
class DictionaryRegister extends Pair<String,String> {
   ...
}
DictionaryRegister[] arr =
              new DictionaryRegister [2];     // OK
```

OU

```
List<Pair<String,String>> list =
new ArrayList <Pair<String,String>>();
```

Referências

- Java Generics Frequently Asked Questions written and maintained by AngelikaLanger

  www.AngelikaLanger.com/GenericsFAQ/JavaGenericsFAQ.html

- Generics in the Java Programming Language by Gilad Bracha

  http://java.sun.com/j2se/1.5/pdf/generics-tutorial.pdf